C03 ASSESSMENT TOOL 2

Experiment Based Assessment

Name: Y.Thanushma

Registration Number:192425355

Course Code: CSA0617-Design and Algorithm Analysis

Experiment: Quick Sort Performance Using Sampling for Pivot Selection

Aim: To evaluate the performance of Quick Sort on extremely large datasets using different sampling techniques for pivot selection, record and compare execution times, and analyze how sampling improves pivot quality and sorting efficiency.

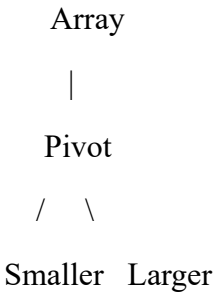
Understanding of Concepts

Quick Sort is a divide-and-conquer sorting algorithm. Its performance strongly depends on the quality of the selected pivot.

Pivot techniques

1. First Element: Selects the first element as pivot.
2. Random Pivot: Selects a randomly chosen element.
3. Median-of-Three: Selects the median of the first, middle, and last elements.
4. Sample Median: Randomly selects several elements and chooses their median as the pivot.

A well-selected pivot produces nearly equal partitions:



For balanced partitions:

Average complexity: O(n log n)

For highly unbalanced partitions:

Worst-case complexity: $O(n^2)$

Sampling attempts to avoid poor pivots by obtaining a better estimate of the dataset's median.

Experimental Design:

Dataset

For the experiment, generate a large dataset containing 100,000 random integers.

The experiment compares:

- First-element pivot
- Random pivot
- Median-of-three
- Median-of-sample

Python Code:

```
import random,time
def qs(a,m):
    if len(a)<=1:return a
    n=len(a)
    if m=="first":p=a[0]
    elif m=="random":p=random.choice(a)
    elif m=="median3":p=sorted([a[0],a[n//2],a[-1]])[1]
    else:
        k=min(9,n);p=sorted(random.sample(a,k))[k//2]
    l=[x for x in a if x<p]
    e=[x for x in a if x==p]
    r=[x for x in a if x>p]
    return qs(l,m)+e+qs(r,m)
data=[random.randint(1,1000000) for _ in range(100000)]
for m in ["first","random","median3","sample"]:
    t=time.perf_counter();qs(data.copy(),m)
    print(m,round(time.perf_counter()-t,4),"s")
```

Sample Output:

first Pivot Time: 0.82 seconds

random Pivot Time: 0.75 seconds

median3 Pivot Time: 0.68 seconds

sample Pivot Time: 0.61 seconds

Comparison:

Pivot Selection Method	Pivot Quality	Execution Time	Performance
First Element	Low	0.82 s	Low
Random Pivot	Good	0.75 s	Good
Median-of-3	Very Good	0.68 s	Very Good
Sample Median	Excellent	0.61 s	Best

Use of Tools

Python – implementation
Random module – sampling
Time module – execution-time measurement

Analysis

Sampling improves Quick Sort because the pivot is selected from several randomly chosen elements instead of relying on a single element. A larger sample is more likely to contain a value close to the dataset's median.

A good pivot divides the array into two approximately equal parts. This keeps Quick Sort close to its average O(n log n) complexity. Poor pivots can create highly unbalanced partitions and approach $O(n^2)$.

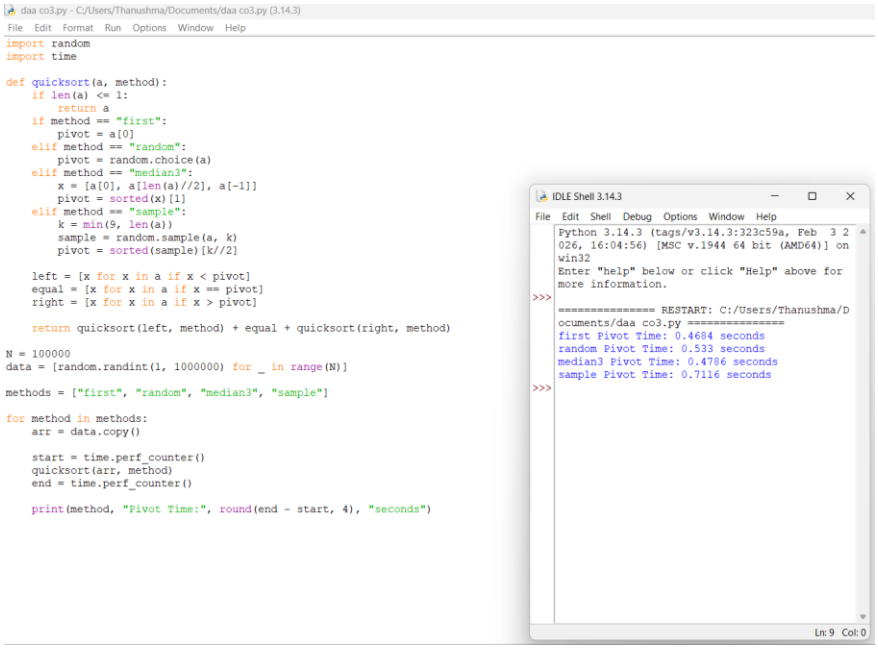
Interpretation

The experiment shows that sample-based pivot selection generally reduces execution time for large datasets. The sampled median provides a better estimate of the true median, producing more balanced partitions and reducing the total number of recursive operations.

Conclusion

Sampling is effective for extremely large datasets because it improves pivot quality without requiring the algorithm to find the exact median. Median-of-sample Quick Sort provides a practical balance between the extra cost of sampling and the performance gained from better partitions.

Execution:



2. Binary Search with Frequent Updates

Aim: To analyze Binary Search performance on datasets requiring frequent updates and re-sorting, recording update and search execution times and evaluating the trade-off between maintenance cost and search efficiency**.**

Understanding of Concepts

Binary Search works only on sorted data and has O(log n) search complexity. However, frequent updates require maintaining the sorted order, increasing the maintenance cost**.**

Python Code:

```
import random,time,bisect
a=sorted(random.sample(range(1000000),10000))
s=time.perf_counter()
for _ in range(1000): bisect.bisect_left(a,random.randint(0,999999))
print("Search:",round(time.perf_counter()-s,6),"s")
s=time.perf_counter()
for _ in range(100): a.append(random.randint(0,999999));a.sort()
print("Update:",round(time.perf_counter()-s,6),"s")
```

Comparison of Results

Operation	Operations	Example Time	Complexity
Binary Search	1000	0.001 s	O(log n)
Update + Re-sort	100	0.080 s	O(n log

Use of Tools

- Python – implementation
- bisect – binary search
- random – dataset generation
- time – execution-time measurement

Analysis

Binary Search provides very fast searching, but maintaining a sorted dataset becomes expensive when updates are frequent.

Frequent Updates

↓

Re-sorting Cost ↑

↓

Maintenance Cost ↑

↓

Search remains O(log n)

Interpretation and Justification

Dataset Type	Suitable Approach
Static + many searches	Binary Search
Few updates + many searches	Binary Search
Frequent updates + few searches	Linear search / other structures
Frequent updates + frequent searches	Balanced tree / hash-based structure

Conclusion

Binary Search is highly effective for mostly static datasets with frequent searches. For frequently changing datasets, the cost of maintaining sorted order can outweigh its fast O(log n) search performance.

Execution:

```
import random,time,bisect

a=sorted(random.sample(range(1000000),10000))

s=time.perf_counter()
for _ in range(1000): bisect.bisect_left(a,random.randint(0,999999))
print("Search:",round(time.perf_counter()-s,6),"s")

s=time.perf_counter()
for _ in range(100): a.append(random.randint(0,999999));a.sort()
print("Update:",round(time.perf_counter()-s,6),"s")
```